DEOS CAN-FD Interface Control Document (ICD)

Sürüm 0.3

DEOS – Dokuz Eylül Otonom Sistemler

Doküman tipi:	Interface Control Document (ICD) – **ARŞİV**
Protokol sürümü:	v1.1 (`0x11`)
Taşıma altyapısı:	CAN-FD ana omurga
Durum:	ICD Sürüm 0.3 (Monolitik Arşiv – Güncel Değil)

ARŞİV DOKÜMANI / DEPRECATION NOTICE:
Bu tek parça (monolitik) ICD dokümanı (**Sürüm 0.3**), DEOS protokolünün ilk tasarım evresine ait olup geriye dönük referans amacıyla arşivlenmiştir.
Güncel, modüler ve resmi protokol şartnamesi `docs/icd/` dizininde yer alan **Sürüm 1.1** doküman ailesidir (`DOC-ICD-CORE-001` ile `DOC-ICD-NODE-BMS-012` arası). Yeni geliştirme ve entegrasyon süreçlerinde modüler doküman setini esas alınız.

1 Amaç ve Kapsam

Bu doküman, DEOS araç içi CAN-FD haberleşme hattında kullanılan ortak adresleme, 29-bit Extended CAN Identifier yapısı, mesaj sınıfları, servisler, komut namespace'leri, ortak veri alanı, veri tipleri ve temel mesaj formatlarını tanımlar.

Dokümanın kapsamı CAN hattında gözlenebilen protokol davranışlarıdır. Node ID, Priority, Message Class, Service, Source, Destination, Command, Protocol Version, Sequence, Length ve Command Payload alanlarının wire-format karşılıkları normatif olarak tanımlanır.

Gömülü yazılım mimarisi ve CAN-FD hattı dışındaki taşıma ortamlarının çalışma ayrıntıları bu ICD'nin kapsamı dışındadır.

Bir Node ID fiziksel bir mikrodenetleyiciye birebir bağlı olmak zorunda değildir. Aynı fiziksel cihaz üzerinde birden fazla mantıksal DEOS Node ID bulunabilir; ancak CAN hattında her Node ID bağımsız bir protokol kimliği olarak değerlendirilir.

2 Namespace Hiyerarşisi

DEOS protokolünde bir mesajın kimliği tek bir "mesaj numarası" ile değil, birden fazla alanın birleşimiyle belirlenir:

```
DEOS_CAN_PROTOCOL
|
+-- NODE
|   +-- SOURCE_NODE
|   +-- DESTINATION_NODE
|
+-- MESSAGE_CLASS
|
+-- SERVICE
|   +-- COMMAND
|       +-- PAYLOAD
|           +-- PARAMETER_ID   (gerektiğinde)
|           +-- VALUE          (gerektiğinde)
|
```

```
+-- PROTOCOL_METADATA
    +-- VERSION
    +-- SEQUENCE_RESERVED
    +-- LENGTH
```

Buna göre bir mesajın semantik kimliği genel olarak şu bileşenlerden oluşur:

`Priority + Message Class + Service + Destination + Source + Command`

Parametre yönetimi yapılan mesajlarda buna ek olarak `Parameter ID` kullanılır.

3 CAN-FD Extended Identifier Yapısı

DEOS protokolü 29-bit Extended CAN Identifier kullanır. Identifier alanı aşağıdaki şekilde bölünür:

Bitler	Alan	Uzunluk	Açıklama
28–26	Priority	3 bit	Arbitration önceliği
25–22	Message Class	4 bit	Mesaj davranış sınıfı
21–16	Service ID	6 bit	Fonksiyonel servis alanı
15–8	Destination Node	8 bit	Hedef düğüm adresi
7–0	Source Node	8 bit	Kaynak düğüm adresi

Toplam uzunluk:

$$3 + 4 + 6 + 8 + 8 = 29 \text{ bit}$$

Identifier sayısal olarak aşağıdaki bağıntı ile ifade edilir:

$$\text{CAN_ID} = (P \ll 26) \mid (C \ll 22) \mid (S \ll 16) \mid (D \ll 8) \mid SRC$$

Burada:

- P: Priority,
- C: Message Class,
- S: Service ID,
- D: Destination Node,
- SRC: Source Node değeridir.

CAN arbitration mantığı nedeniyle sayısal olarak daha düşük identifier daha yüksek önceliğe sahiptir. Bu nedenle Priority alanında düşük sayısal değer daha yüksek önceliği temsil eder.

3.1 Önerilen Priority Registry

Değer	Symbol	Kullanım
0	`DEOS_PRIO_EMERGENCY`	Acil durdurma ve en kritik safety mesajları
1	`DEOS_PRIO_CRITICAL_CONTROL`	Kritik gerçek zamanlı kontrol
2	`DEOS_PRIO_CONTROL`	Normal sürüş kontrol mesajları
3	`DEOS_PRIO_STATUS`	Durum ve geri besleme mesajları
4	`DEOS_PRIO_CONFIG`	Konfigürasyon ve kalibrasyon
5	`DEOS_PRIO_NETWORK`	Network management / heartbeat
6	`DEOS_PRIO_LOW`	Düşük öncelikli servis trafiği
7	`DEOS_PRIO_BACKGROUND`	Arka plan / geliştirme trafiği

4 Node Address Registry

Her fiziksel veya sanal DEOS haberleşme düğümüne sistem genelinde benzersiz bir 8-bit Logical Node ID atanır. Node ID, düğümün fiziksel donanım adresi veya transport adresi değildir; protokol içerisindeki mantıksal kimliğidir.

4.1 Adres Aralıkları

Adres Aralığı	Kullanım
`0x00`	Invalid / Unassigned
`0x01`–`0x0F`	Merkezi sistem ve virtual node'lar
`0x10`–`0x1F`	Aktüatör / motor ECU düğümleri
`0x20`–`0x2F`	Güç sistemi / BMS düğümleri
`0x30`–`0x3F`	Gateway / protocol bridge düğümleri
`0x40`–`0x5F`	Sensör düğümleri için ayrılmış alan
`0x60`–`0xDF`	Gelecek genişleme
`0xE0`–`0xEF`	Geliştirme / test düğümleri
`0xF0`–`0xFE`	Diagnostik ve servis araçları
`0xFF`	Broadcast

4.2 Virtual Node Atamaları

Virtual node, bağımsız bir fiziksel mikrodenetleyici veya bilgisayar olmak zorunda olmayan, ancak DEOS protokolünde kendi Source ve Destination Node ID'si ile adreslenebilen mantıksal haberleşme ucudur.

Node ID	Symbol	Çalıştığı Ortam	Görev
`0x01`	`DEOS_NODE_TEXTUAL`	Raspberry Pi / Textual	Operatör arayüzü; config, command, request/response, status ve log sink
`0x02`	`DEOS_NODE_GROUND_CONTROL`	Uzak yer kontrol uygulaması	Yer kontrol haberleşme ucu
`0x04`	`DEOS_NODE_MICRO_ROS`	Raspberry Pi / micro-ROS	ROS arayüzü, telemetry ve control entegrasyonu

Textual Node yalnızca konfigürasyon amacıyla kullanılan bir node değildir. Protokolün izin verdiği servislerde komut gönderebilir, cevap alabilir, durum bilgilerini işleyebilir ve log mesajlarının doğrudan hedefi olabilir. Benzer şekilde micro-ROS ve Ground Control Node'ları da ihtiyaç duydukları servisleri kaynak veya hedef olarak kullanabilir.

4.3 Physical Node Atamaları

Node ID	Symbol	Fiziksel Cihaz	Görev
`0x03`	`DEOS_NODE_MAIN_STM32`	STM32H563ZI	Ana fiziksel kontrol düğümü
`0x10`	`DEOS_NODE_TRACTION`	STM32 Traction ECU	Tahrik sistemi kontrolü
`0x11`	`DEOS_NODE_STEERING`	STM32 Steering ECU	Direksiyon sistemi kontrolü
`0x12`	`DEOS_NODE_BRAKE`	STM32 Brake ECU	Fren sistemi kontrolü
`0x20`	`DEOS_NODE_BMS_MAIN`	DALY BMS #1	Ana batarya BMS
`0x21`	`DEOS_NODE_BMS_AUX`	DALY BMS #2	Yardımcı batarya BMS
`0x30`	`DEOS_NODE_BMS_GATEWAY`	CAN Gateway	Classic CAN–CAN-FD çevirici

4.4 Servis ve Özel Node Atamaları

Node ID	Symbol	Tür	Görev
0xF0	DEOS_NODE_DIAG_TOOL	Servis aracı	Diagnostik ve servis erişimi
0xFF	DEOS_NODE_BROADCAST	Mantıksal	Tüm düğümlere yayın

4.5 Node Enum Tanımı

```
typedef enum
{
    DEOS_NODE_INVALID         = 0x00,

    DEOS_NODE_TEXTUAL         = 0x01,
    DEOS_NODE_GROUND_CONTROL  = 0x02,
    DEOS_NODE_MAIN_STM32      = 0x03,
    DEOS_NODE_MICRO_ROS       = 0x04,

    DEOS_NODE_TRACTION        = 0x10,
    DEOS_NODE_STEERING        = 0x11,
    DEOS_NODE_BRAKE           = 0x12,

    DEOS_NODE_BMS_MAIN        = 0x20,
    DEOS_NODE_BMS_AUX         = 0x21,

    DEOS_NODE_BMS_GATEWAY     = 0x30,

    DEOS_NODE_DIAG_TOOL       = 0xF0,
    DEOS_NODE_BROADCAST       = 0xFF
} deos_node_id_t;
```

5 Message Class Registry

Message Class alanı mesajın hangi amaçla gönderildiğini tanımlar. Message Class, fonksiyonel servisten bağımsızdır.

ID	Symbol	Açıklama
0x0	DEOS_CLASS_SAFETY	Acil / safety-critical mesaj
0x1	DEOS_CLASS_COMMAND	Normal gerçek zamanlı kontrol komutu
0x2	DEOS_CLASS_STATUS	Durum / geri besleme / telemetri
0x3	DEOS_CLASS_EVENT	Fault veya olay bildirimi
0x4	DEOS_CLASS_REQUEST	Cevap beklenen okuma, sorgulama veya işlem isteği
0x5	DEOS_CLASS_RESPONSE	Bir REQUEST veya cevap gerektiren işleme verilen protokol cevabı
0x6	DEOS_CLASS_CONFIG	Parametre ve konfigürasyon işlemleri
0x7	DEOS_CLASS_NETWORK	Ağ yönetimi / heartbeat
0x8–0xF	Reserved	Gelecek kullanım

```
typedef enum
{
    DEOS_CLASS_SAFETY   = 0x0,
    DEOS_CLASS_COMMAND  = 0x1,
    DEOS_CLASS_STATUS   = 0x2,
    DEOS_CLASS_EVENT    = 0x3,
    DEOS_CLASS_REQUEST  = 0x4,
    DEOS_CLASS_RESPONSE = 0x5,
    DEOS_CLASS_CONFIG   = 0x6,
    DEOS_CLASS_NETWORK  = 0x7
} deos_message_class_t;
```

5.1 Message Class ve Service Arasındaki Fark

Message Class ile Service farklı sorulara cevap verir:

Alan	Cevapladığı soru	Örnek
Message Class	Mesaj node içerisinde hangi tür işlem akışına aittir?	COMMAND, CONFIG, NETWORK, RESPONSE
Service	Mesaj hangi fonksiyonel alana aittir?	STEERING, BRAKE, BMS, SYSTEM
Command	Bu fonksiyonel alan içerisinde tam olarak hangi işlem yapılacaktır?	SET_TARGET_ANGLE, SET_PARAMETER, PING

Aynı Service farklı Message Class'lar altında kullanılabilir. Örneğin STEERING servisi için:

```
COMMAND / STEERING / SET_TARGET_ANGLE
CONFIG  / STEERING / SET_PARAMETER
REQUEST / STEERING / GET_STATUS
```

Burada Service her üç mesajda da direksiyon fonksiyonel alanını belirtirken Message Class, mesajların node içerisinde farklı işlem yollarına ayrılmasını sağlar.

Tersi de mümkündür: aynı Message Class farklı Service'lerde kullanılabilir:

```
COMMAND / TRACTION / SET_TARGET_SPEED
COMMAND / STEERING / SET_TARGET_ANGLE
COMMAND / BRAKE    / SET_TARGET
```

Bu örneklerde Message Class tüm mesajların normal kontrol komutu olduğunu, Service ise hangi alt sisteme ait olduklarını belirtir.

6 Service Registry

Service alanı mesajın hangi fonksiyonel domaine ait olduğunu gösterir.

Service ID	Symbol	Amaç
0x00	DEOS_SERVICE_SYSTEM	Genel node state/mode/reset bilgileri
0x01	DEOS_SERVICE_TRACTION	Tahrik sistemi
0x02	DEOS_SERVICE_STEERING	Direksiyon sistemi
0x03	DEOS_SERVICE_BRAKE	Fren sistemi
0x04	DEOS_SERVICE_BMS	Batarya ve BMS verileri
0x05	DEOS_SERVICE_CALIBRATION	Kalibrasyon işlemleri
0x06	DEOS_SERVICE_DIAGNOSTIC	Fault / sağlık / diagnostik
0x07	DEOS_SERVICE_SENSORS	Sensör ve IMU veri servisi
0x08	DEOS_SERVICE_LIGHTING	Aydınlatma ve sinyalizasyon sistemi
0x09–0x3F	Reserved	Yeni servisler için ayrılmış alan

```
typedef enum
{
    DEOS_SERVICE_SYSTEM      = 0x00,
    DEOS_SERVICE_TRACTION    = 0x01,
    DEOS_SERVICE_STEERING    = 0x02,
    DEOS_SERVICE_BRAKE       = 0x03,
    DEOS_SERVICE_BMS         = 0x04,
    DEOS_SERVICE_CALIBRATION = 0x05,
    DEOS_SERVICE_DIAGNOSTIC  = 0x06,
    DEOS_SERVICE_SENSORS     = 0x07,
    DEOS_SERVICE_LIGHTING    = 0x08
} deos_service_id_t;
```

7 Node–Service İlişkisi

Node ve Service aynı kavram değildir. Node, fiziksel veya sanal haberleşme düğümünü; Service ise mesajın ait olduğu fonksiyonel alanı tanımlar. Bir node'un belirli bir servise mesaj gönderebilmesi, o servisin kendisi üzerinde barındırıldığı anlamına gelmez.

Aşağıdaki tablo temel service handler sahipliğini gösterir. Her adreslenebilir DEOS node'u, ayrıca bu ICD'de tanımlanan ortak SYSTEM/NETWORK PING–PONG davranışını desteklemek zorundadır.

Node	SYS	TRAC	STEER	BRAKE	BMS	CAL	DIAG	LIGHT
TEXTUAL	X	–	–	–	–	–	X	–
GROUND_CONTROL	X	–	–	–	–	–	X	–
MAIN_STM32	X	X	X	X	X	X	X	X
MICRO_ROS	X	–	–	–	–	–	X	–
TRACTION	X	X	–	–	–	X	X	–
STEERING	X	–	X	–	–	X	X	–
BRAKE	X	–	–	X	–	X	X	–
BMS_MAIN	X	–	–	–	X	–	X	–
BMS_AUX	X	–	–	–	X	–	X	–
BMS_GATEWAY	X	–	–	–	X	–	X	–

Tablodaki işaretler, ilgili node'un o servise ait mesajları desteklediğini ifade eder; mesaj gönderme yeteneğini sınırlamaz. Örneğin `DEOS_NODE_TEXTUAL`, hedefi `DEOS_NODE_STEERING` olan ve `DEOS_SERVICE_STEERING` kullanan bir CONFIG veya REQUEST mesajı gönderebilir.

Örneğin direksiyon düğümüne normal sürüş komutu gönderilirken `Destination = DEOS_NODE_STEERING`, `Service = DEOS_SERVICE_STEERING` kullanılır. Aynı fiziksel düğüme kalibrasyon komutu gönderilirken `Destination = DEOS_NODE_STEERING`, `Service = DEOS_SERVICE_CALIBRATION` kullanılır.

8 Command Namespace

Command ID 8-bit olarak tanımlanır. Command ID tek başına global değildir; `Service ID + Command ID` birleşimi komutu benzersiz hale getirir.

Genel aralık politikası:

Command Aralığı	Kullanım
`0x00`	Invalid / Reserved
`0x01–0x7F`	Service-specific komutlar
`0x80–0xDF`	Gelecek service-specific genişleme
`0xE0–0xE4`	Ortak Parameter Registry komutları
`0xE5–0xEF`	Ortak yönetim komutları için reserved
`0xF0–0xFE`	Geliştirme / deneysel
`0xFF`	Reserved

8.1 SYSTEM Service Komutları

ID	Symbol
0x01	DEOS_CMD_SYSTEM_GET_STATE
0x02	DEOS_CMD_SYSTEM_SET_STATE
0x03	DEOS_CMD_SYSTEM_GET_MODE
0x04	DEOS_CMD_SYSTEM_SET_MODE
0x05	DEOS_CMD_SYSTEM_RESET_NODE
0x06	DEOS_CMD_SYSTEM_GET_NODE_INFO
0x07	DEOS_CMD_SYSTEM_HEARTBEAT
0x08	DEOS_CMD_SYSTEM_PING
0x09	DEOS_CMD_SYSTEM_PONG

```
typedef enum
{
    DEOS_CMD_SYSTEM_GET_STATE      = 0x01,
    DEOS_CMD_SYSTEM_SET_STATE      = 0x02,
    DEOS_CMD_SYSTEM_GET_MODE       = 0x03,
    DEOS_CMD_SYSTEM_SET_MODE       = 0x04,
    DEOS_CMD_SYSTEM_RESET_NODE     = 0x05,
    DEOS_CMD_SYSTEM_GET_NODE_INFO  = 0x06,
    DEOS_CMD_SYSTEM_HEARTBEAT      = 0x07,
    DEOS_CMD_SYSTEM_PING           = 0x08,
    DEOS_CMD_SYSTEM_PONG           = 0x09
} deos_system_command_t;
```

8.2 TRACTION Service Komutları

ID	Symbol
0x01	DEOS_CMD_TRACTION_SET_TARGET_SPEED
0x02	DEOS_CMD_TRACTION_GET_STATUS

```
typedef enum
{
    DEOS_CMD_TRACTION_SET_TARGET_SPEED = 0x01,
    DEOS_CMD_TRACTION_GET_STATUS       = 0x02
} deos_traction_command_t;
```

8.3 STEERING Service Komutları

ID	Symbol
0x01	DEOS_CMD_STEERING_SET_TARGET_ANGLE
0x02	DEOS_CMD_STEERING_GET_STATUS

```
typedef enum
{
    DEOS_CMD_STEERING_SET_TARGET_ANGLE = 0x01,
    DEOS_CMD_STEERING_GET_STATUS       = 0x02
} deos_steering_command_t;
```

8.4 BRAKE Service Komutları

ID	Symbol
0x01	DEOS_CMD_BRAKE_SET_TARGET
0x02	DEOS_CMD_BRAKE_GET_STATUS

```
typedef enum
{
    DEOS_CMD_BRAKE_SET_TARGET = 0x01,
    DEOS_CMD_BRAKE_GET_STATUS = 0x02
} deos_brake_command_t;
```

8.5 BMS Service Komutları

ID	Symbol
0x01	DEOS_CMD_BMS_GET_STATUS
0x02	DEOS_CMD_BMS_STATUS

```
typedef enum
{
    DEOS_CMD_BMS_GET_STATUS = 0x01,
    DEOS_CMD_BMS_STATUS     = 0x02
} deos_bms_command_t;
```

8.6 CALIBRATION Service Komutları

ID	Symbol
0x01	DEOS_CMD_CAL_START
0x02	DEOS_CMD_CAL_ABORT
0x03	DEOS_CMD_CAL_GET_STATUS
0x04	DEOS_CMD_CAL_SAVE_RESULT
0x05	DEOS_CMD_CAL_CLEAR_RESULT

```
typedef enum
{
    DEOS_CMD_CAL_START        = 0x01,
    DEOS_CMD_CAL_ABORT        = 0x02,
    DEOS_CMD_CAL_GET_STATUS   = 0x03,
    DEOS_CMD_CAL_SAVE_RESULT  = 0x04,
    DEOS_CMD_CAL_CLEAR_RESULT = 0x05
} deos_calibration_command_t;
```

8.7 DIAGNOSTIC Service Komutları

ID	Symbol
0x01	DEOS_CMD_DIAG_GET_HEALTH
0x02	DEOS_CMD_DIAG_GET_FAULTS
0x03	DEOS_CMD_DIAG_CLEAR_FAULTS

```
typedef enum
{
    DEOS_CMD_DIAG_GET_HEALTH   = 0x01,
    DEOS_CMD_DIAG_GET_FAULTS   = 0x02,
    DEOS_CMD_DIAG_CLEAR_FAULTS = 0x03
} deos_diagnostic_command_t;
```

8.8 LIGHTING Service Komutları

Aydınlatma servisi (`DEOS_SERVICE_LIGHTING = 0x08`) araçtaki farların açılıp kapatılmasını yönetir.

ID	Symbol	Length	Açıklama
0x01	DEOS_CMD_LIGHTING_TURN_ON	1 byte	Hedef farı açar (Payload: Light ID)
0x02	DEOS_CMD_LIGHTING_TURN_OFF	1 byte	Hedef farı kapatır (Payload: Light ID)

```
typedef enum
{
    DEOS_CMD_LIGHTING_TURN_ON  = 0x01,
    DEOS_CMD_LIGHTING_TURN_OFF = 0x02
} deos_lighting_command_t;
```

8.8.1 Far ID Tanımları (Light ID)

Komut payload'ında hedef far 1 baytlık `Light ID` ile iletilir:

Adres	Sembolik İsim	Açıklama
0x01	DEOS_LIGHT_HEADLIGHT	Ön Farlar
0x02	DEOS_LIGHT_BRAKE	Fren Lambası
0x03	DEOS_LIGHT_TURN_LEFT	Sol Sinyal
0x04	DEOS_LIGHT_TURN_RIGHT	Sağ Sinyal
0xFF	DEOS_LIGHT_ALL	Tüm Işıklar

```
typedef enum
{
    DEOS_LIGHT_HEADLIGHT  = 0x01,
    DEOS_LIGHT_BRAKE      = 0x02,
    DEOS_LIGHT_TURN_LEFT  = 0x03,
    DEOS_LIGHT_TURN_RIGHT = 0x04,
    DEOS_LIGHT_ALL        = 0xFF
} deos_light_id_t;
```

Payload toplam 1 bayttır (`Byte 0 = Light ID`, `Length = 0x01`).

9 CONFIG Message Class ve Parameter Registry

Konfigürasyon işlemleri için ayrı bir CONFIG servisi kullanılmaz. Bunun yerine `Message Class = CONFIG` seçilir ve hedef fonksiyonel servis korunur. REQUEST ve RESPONSE sınıfları ise protokolde ayrıca bulunmaya devam eder; CONFIG bunların yerine geçen bir sınıf değildir.

Örneğin direksiyon PID katsayısı değiştirilirken:

```
Destination   = DEOS_NODE_STEERING
Message Class = DEOS_CLASS_CONFIG
Service       = DEOS_SERVICE_STEERING
Command       = DEOS_CMD_SET_PARAMETER
Parameter ID  = DEOS_PARAM_STEERING_PID_KP
```

Bu yapı sayesinde konfigürasyon işlemi hangi alt sisteme ait olduğunu kaybetmez.

9.1 Ortak Parameter Command ID'leri

Aşağıdaki command ID'leri configurable servislerde ortak olarak kullanılır:

ID	Symbol
0xE0	DEOS_CMD_GET_PARAMETER
0xE1	DEOS_CMD_SET_PARAMETER
0xE2	DEOS_CMD_SAVE_PARAMETERS
0xE3	DEOS_CMD_RESTORE_DEFAULTS
0xE4	DEOS_CMD_GET_PARAMETER_INFO

```
typedef enum
{
    DEOS_CMD_GET_PARAMETER        = 0xE0,
    DEOS_CMD_SET_PARAMETER        = 0xE1,
    DEOS_CMD_SAVE_PARAMETERS      = 0xE2,
    DEOS_CMD_RESTORE_DEFAULTS     = 0xE3,
    DEOS_CMD_GET_PARAMETER_INFO   = 0xE4
} deos_parameter_command_t;
```

9.2 Parameter ID Alanı

Parameter ID 16-bit olarak tanımlanır. Parameter ID global değildir; `Service ID + Parameter ID` birlikte benzersizdir.

Örnek Steering Parameter Registry:

ID	Symbol	Açıklama
0x0001	DEOS_PARAM_STEERING_PID_KP	PID oransal katsayısı
0x0002	DEOS_PARAM_STEERING_PID_KI	PID integral katsayısı
0x0003	DEOS_PARAM_STEERING_PID_KD	PID türev katsayısı
0x0010	DEOS_PARAM_STEERING_ZERO_OFFSET	Direksiyon sıfır ofseti
0x0011	DEOS_PARAM_STEERING_MIN_ANGLE	Minimum izinli açı
0x0012	DEOS_PARAM_STEERING_MAX_ANGLE	Maksimum izinli açı
0x0013	DEOS_PARAM_STEERING_CURRENT_LIMIT	Akım limiti

Örnek Traction Parameter Registry:

ID	Symbol	Açıklama
0x0001	DEOS_PARAM_TRACTION_PID_KP	PID oransal katsayısı
0x0002	DEOS_PARAM_TRACTION_PID_KI	PID integral katsayısı
0x0003	DEOS_PARAM_TRACTION_PID_KD	PID türev katsayısı
0x0010	DEOS_PARAM_TRACTION_MAX_CURRENT	Maksimum akım
0x0011	DEOS_PARAM_TRACTION_MAX_SPEED	Maksimum hız
0x0012	DEOS_PARAM_TRACTION_ACCEL_LIMIT	İvme limiti

Örnek Brake Parameter Registry:

ID	Symbol	Açıklama
0x0001	DEOS_PARAM_BRAKE_PID_KP	Gelecek kullanım / gerekli olması halinde
0x0002	DEOS_PARAM_BRAKE_PID_KI	Gelecek kullanım / gerekli olması halinde
0x0003	DEOS_PARAM_BRAKE_PID_KD	Gelecek kullanım / gerekli olması halinde
0x0010	DEOS_PARAM_BRAKE_MIN_POSITION	Minimum aktüatör konumu
0x0011	DEOS_PARAM_BRAKE_MAX_POSITION	Maksimum aktüatör konumu
0x0012	DEOS_PARAM_BRAKE_CURRENT_LIMIT	Akım limiti

10 State ve Mode Registry

State, düğümün anlık çalışma durumunu; Mode ise düğümün hangi çalışma davranışı altında olduğunu ifade eder. İki alan birbirinden bağımsızdır.

10.1 State Enum

Değer	Symbol
0x00	DEOS_STATE_INIT
0x01	DEOS_STATE_STANDBY
0x02	DEOS_STATE_READY
0x03	DEOS_STATE_ACTIVE
0x04	DEOS_STATE_CALIBRATING
0x05	DEOS_STATE_SAFE
0x06	DEOS_STATE_FAULT
0xFF	DEOS_STATE_UNKNOWN

```
typedef enum
{
    DEOS_STATE_INIT         = 0x00,
    DEOS_STATE_STANDBY      = 0x01,
    DEOS_STATE_READY        = 0x02,
    DEOS_STATE_ACTIVE       = 0x03,
    DEOS_STATE_CALIBRATING  = 0x04,
    DEOS_STATE_SAFE         = 0x05,
    DEOS_STATE_FAULT        = 0x06,
    DEOS_STATE_UNKNOWN      = 0xFF
} deos_state_t;
```

10.2 Mode Enum

Değer	Symbol
0x00	DEOS_MODE_NORMAL
0x01	DEOS_MODE_MANUAL
0x02	DEOS_MODE_AUTONOMOUS
0x03	DEOS_MODE_SERVICE
0x04	DEOS_MODE_TEST
0xFF	DEOS_MODE_UNKNOWN

```
typedef enum
{
    DEOS_MODE_NORMAL      = 0x00,
    DEOS_MODE_MANUAL      = 0x01,
```

```
    DEOS_MODE_AUTONOMOUS = 0x02,
    DEOS_MODE_SERVICE    = 0x03,
    DEOS_MODE_TEST       = 0x04,
    DEOS_MODE_UNKNOWN    = 0xFF
} deos_mode_t;
```

11 Uygulama Veri Alanı

DEOS CAN-FD data alanında Source, Destination, Service ve Priority tekrar taşınmaz; bu alanlar 29-bit CAN Identifier içerisinde bulunur. Uygulama seviyesinde ek senkronizasyon kelimesi veya ek CRC tanımlanmaz.

DEOS v1.1 ortak veri alanı aşağıdaki gibidir:

Byte	Alan	Uzunluk	Açıklama
0	Protocol Version	1 byte	Protokol major/minor sürümü
1	Sequence / Reserved	1 byte	Source Node başına TX sequence değeri
2	Command ID	1 byte	İlgili Service namespace'i içindeki komut
3	Length	1 byte	Geçerli command-specific payload byte sayısı
4...N	Command Payload	Değişken	Komuta özgü veri

11.1 Length Alanı

Length alanı `uint8_t` olarak tanımlanır ve yalnızca command-specific payload'un geçerli byte sayısını belirtir. Version, Sequence, Command ve Length alanlarının kendisi bu değere dahil değildir. Geçerli aralık `0..60` byte'tır.

CAN-FD DLC yuvarlaması nedeniyle oluşan padding byte'ları Length değerine dahil değildir ve command payload olarak yorumlanmaz.

Maksimum CAN-FD data alanı 64 byte ve ortak DEOS header'ı 4 byte olduğundan maksimum command-specific payload uzunluğu 60 byte'tır.

11.2 Protocol Version

Version byte iki adet 4-bit alana ayrılır:

Bitler	Anlam
7–4	Major Version
3–0	Minor Version

Aktif wire-format sürümü:

$$\text{DEOS Protocol v1.1} = \texttt{0x11}$$

`0x10` değeri v1.0 eski wire-formatını, `0x11` değeri ise Length alanını içeren v1.1 wire-formatını ifade eder. Bu iki wire-format birbiriyle uyumlu değildir. v1.1 formatında geçerli bir frame'in Protocol Version alanı `0x11` olmalıdır.

11.3 Sequence Alanı

Sequence alanı 8-bit'tir. Her Source Node kimliği kendi transmit sequence değerini kullanır. Sayaç `0xFF` sonrasında `0x00` değerine dönebilir.

v1.1'de Sequence alanı protokol geçerliliği açısından bilgilendirici/reserved semantiğini korur. Receiver, sequence atlamasını veya tekrarını zorunlu olarak paket kaybı, duplicate frame veya protokol hatası olarak yorumlamak zorunda değildir.

12 Veri Tipleri ve Byte Sırası

DEOS protokolünde çok baytlı alanlar için varsayılan byte sırası **Little-Endian** olarak tanımlanır.

Wire formatında temel tercih, doğrudan IEEE-754 floating-point taşımak yerine ölçekli integer değer kullanmaktır. Böylece bir sinyalin fiziksel karşılığı açık şekilde `raw value + scale + offset + unit` ile tanımlanır.

Genel fiziksel değer dönüşümü:

$$PhysicalValue = RawValue \times Scale + Offset$$

12.1 Normatif Veri Tipleri

Tip	Boyut	Kullanım
`uint8_t`	1 byte	Enum, yüzde, küçük unsigned değerler
`int8_t`	1 byte	Küçük signed değerler
`uint16_t`	2 byte	Ölçekli pozitif fiziksel değerler
`int16_t`	2 byte	Ölçekli signed fiziksel değerler
`uint32_t`	4 byte	Sayaç / geniş pozitif değerler
`int32_t`	4 byte	Geniş signed fiziksel değerler
`float32`	4 byte	Yalnız ilgili mesaj tanımında açıkça belirtilirse IEEE-754

Floating-point kullanımına genel yasak konulmamıştır; ancak bir alan `float32` olarak tanımlanmamışsa integer tabanlı ölçekleme varsayılır.

12.2 Başlangıç Fiziksel Sinyal Formatları

Sinyal	Wire Type	Scale	Unit	Örnek
Steering target angle	`int16_t`	0.01	degree	1250 = 12.50 deg
Vehicle target speed	`uint16_t`	0.01	m/s	725 = 7.25 m/s
Brake target	`uint16_t`	0.01	%	7500 = 75.00%
BMS voltage	`uint16_t`	0.01	V	7425 = 74.25 V
BMS current	`int16_t`	0.1	A	-125 = -12.5 A
BMS SOC	`uint16_t`	0.01	%	8530 = 85.30%
Temperature	`int16_t`	0.1	degC	253 = 25.3 degC

Bu tablo başlangıç wire-format standardını tanımlar. Nihai min/max sınırlar ve invalid değerler ilgili komut veya sinyal tanımında ayrıca belirtilmelidir.

13 CAN-FD DLC, Payload Uzunluğu ve BRS

CAN-FD DLC alanı, frame'in fiziksel CAN-FD data field boyutunu belirtir. DEOS v1.1 Length alanı ise yalnızca geçerli command-specific payload byte sayısını belirtir. Bu iki alan aynı anlamda değildir.

8 byte üzerindeki CAN-FD data alanları 12, 16, 20, 24, 32, 48 veya 64 byte fiziksel boyutlara yuvarlanabilir. Bu nedenle DEOS Length değeri fiziksel CAN-FD data field boyutundan bağımsız olarak taşınır.

Örnek olarak 10 byte command payload için:

```
DEOS common header  = 4 byte
Command payload     = 10 byte
Semantic data size  = 14 byte
CAN-FD data size    = 16 byte
Padding             = 2 byte
```

DLC yuvarlaması nedeniyle semantic data sonrasında kalan kullanılmayan byte'lar `0x00` olarak gönderilir ve alıcı tarafından payload'ın parçası olarak değerlendirilmez.

DEOS CAN-FD frame'leri 29-bit Extended Identifier, CAN-FD formatı ve Bit Rate Switching (BRS) kullanır. Nominal/arbitration bitrate ile data-phase bitrate değerleri deployment parametreleridir ve bu ICD revizyonunda sabitlenmemiştir.

14 Medya Bağımsız Standart DEOS Taşıma Çerçevesi (Universal Transport Frame)

DEOS protokolü, CAN-FD donanımının 29-bit Arbitration ID ve bit paketleme kısıtlarından bağımsız olarak; Ethernet (TCP/UDP), LoRa (UART), ROS 2 köprüleri ve Yer Kontrol İstasyonu (Python) gibi tüm harici sistemlerle uçtan uca haberleşebilmek için standart bir **Evrensel Taşıma Çerçevesi (Universal DEOS Transport Frame)** tanımlar.

Dış ortamdan üretilen bir mesaj, CAN ID hesaplamasına veya donanımsal bit kaydırmalarına gerek kalmaksızın doğrudan bu standart dizilim ile paketlenir. Mesaj STM32 Ana Beyin (Router) katmanına ulaştığında, hedef CAN düğümüyse otomatik olarak 29-bit CAN ID ve CAN-FD verisine dönüştürülür.

14.1 Standart Çerçeve Anatomisi (Byte Byte)

Evrensel çerçeve, **9 Byte Sabit Başlık (Header)** ve ardından gelen N Byte **Payload** alanından oluşur:

Byte	Alan Adı	Tip	Boyut	Açıklama
0	`Destination Node`	`uint8_t`	1 Byte	Hedef düğüm kimliği (Örn: `0x03` Main STM32, `0x10` Traction, `0xFF` Broadcast).
1	`Source Node`	`uint8_t`	1 Byte	Kaynak düğüm kimliği (Örn: `0x02` Ground Control, `0x06` Textual UI).
2	`Priority`	`uint8_t`	1 Byte	Mesaj önceliği (`0x00` CRITICAL .. `0x05` NETWORK).
3	`Version`	`uint8_t`	1 Byte	DEOS protokol sürümü (Varsayılan: `0x11` = v1.1).
4	`Message Class`	`uint8_t`	1 Byte	Mesaj sınıfı (`0x01` COMMAND, `0x02` RESPONSE, `0x07` NETWORK vb.).
5	`Service ID`	`uint8_t`	1 Byte	Fonksiyonel servis (`0x00` SYSTEM, `0x01` TRACTION, `0x08` LIGHTING vb.).
6	`Command ID`	`uint8_t`	1 Byte	Servise ait komut (`0x01` TURN_ON, `0x02` TURN_OFF, `0x08` PING vb.).
7	`Sequence`	`uint8_t`	1 Byte	Mesaj takip sayacı (`0x00` – `0xFF`).
8	`Payload Length`	`uint8_t`	1 Byte	Arkadan gelen dinamik veri bayt sayısı (N, 0–60 byte).
9..	**Payload**	`uint8_t[`N`]`	N Byte	Gerçek komut/telemetri verisi (Örn: Far ID, Ping ID, Hız hedefi).

Toplam Paket Boyutu: $9 + N$ Byte.

14.2 CAN-FD Dönüşüm Mekanizması

Bu çerçeve STM32 Gateway/Router düğümüne ulaştığında:

1. **29-bit CAN ID:** Byte 0 (`Destination`), Byte 1 (`Source`), Byte 2 (`Priority`), Byte 4 (`Message Class`) ve Byte 5 (`Service`) alanları sürücü tarafından otomatik olarak 29-bit Extended CAN Identifier'a dönüştürülür.
2. **CAN-FD Data Payload:** Byte 3 (`Version`), Byte 7 (`Sequence`), Byte 6 (`Command`), Byte 8 (`Payload Length`) ve ardından gelen `Payload` alanları doğrudan CAN-FD frame'inin veri alanına yazılır.

14.3 Fiziksel İletim Katmanlarına Göre Dış Zarf (L2 Framing)

Evrensel 9+N baytlık standart DEOS çerçevesi, fiziksel iletim hattının özelliklerine göre dış zarflarla taşınır:

- **Ethernet (TCP):** TCP zaten akış kontrolü ve sağlama toplamı sağladığından, çerçevenin başına sadece toplam paket boyutu eklenir: `[Length (1B)] + [Standart DEOS Frame (9+N Byte)]`.
- **LoRa / Seri Port (UART):** Gürültülü ve bayt kayması olabilecek hatlarda senkronizasyon ve CRC doğrulaması eklenir: `[0xAA 0x55] [Flag] [Length] + [Standart DEOS Frame (9+N Byte)] + [CRC16 (2B)]`.
- **ROS 2 / UDP:** Doğrudan `[Standart DEOS Frame (9+N Byte)]` datagram olarak iletilir.

15 REQUEST–RESPONSE ve CONFIG Ayrımı

REQUEST, RESPONSE ve CONFIG birbirinden bağımsız mesaj sınıflarıdır ve farklı amaçlara hizmet eder.

- `REQUEST`: Bir düğümden veri, durum veya belirli bir işlem sonucu talep etmek için kullanılır.
- `RESPONSE`: REQUEST sınıfındaki bir mesaja veya cevap gerektiren başka bir işleme karşılık dönen cevabı taşır.
- `CONFIG`: Bir düğümün çalışma parametrelerini, ayarlarını veya kalıcı yapılandırmasını değiştirmeye yönelik mesajları taşır.

Örneğin bir parametreyi okumak için REQUEST, aynı parametreyi değiştirmek için CONFIG kullanılabilir:

```
REQUEST / STEERING / GET_PARAMETER / PID_KP
CONFIG  / STEERING / SET_PARAMETER / PID_KP
```

Her iki işlem için de gerektiğinde RESPONSE sınıfı ile sonuç döndürülebilir. Bu ayrım, mesajın "cevap bekleyip beklemediği" ile "konfigürasyon amacı taşıyıp taşımadığı" bilgilerinin birbirine karıştırılmasını önler.

16 Response Altyapısı

RESPONSE Message Class protokolde ayrılmıştır; ancak hangi komutların zorunlu response üreteceği bu registry dokümanında kesinleştirilmemiştir.

Özellikle aşağıdaki işlemler gelecekte response kullanılabilecek adaylardır:

- Parameter GET/SET,
- state veya mode değişimi,
- kalibrasyon başlatma / kaydetme,
- fault reset / clear,
- node reset,
- servis tipi tek-seferlik yönetim komutları.

16.1 Generic Response Payload

`DEOS_CLASS_RESPONSE` mesajları varsayılan olarak standart bir veri yerleşimi kullanır. Eğer ilgili komut özel bir ek veri döndürmüyorsa, yanıt mesajının içeriği her zaman şu formatta olur:

Byte	Alan	Tip	Açıklama
0	Result Code	`uint8_t`	İşlem sonucu (`deos_result_t`)
1..N	Ek Veri (Opsiyonel)	Değişken	Komuta özgü veri

Result code `0x00` (SUCCESS) olduğunda işlem başarılı sayılır. Bu genel yapı sayesinde, her komut için özel bir response formatı tasarlamaya gerek kalmadan hata yönetimi standartlaştırılmıştır.

Aşağıda standart response result kodları listelenmiştir:

Kod	Anlam
0x00	SUCCESS
0x01	UNSUPPORTED
0x02	INVALID_PARAMETER
0x03	OUT_OF_RANGE
0x04	NOT_ALLOWED
0x05	BUSY
0x06	NOT_READY
0x07	INTERNAL_ERROR

Bu alanın kesin payload yerleşimi sonraki ICD revizyonunda mesaj bazında ayrıntılandırılacaktır.

17 PING–PONG Ağ Erişilebilirlik Mekanizması

DEOS protokolünde tüm adreslenebilir physical ve virtual node'lar ortak bir PING–PONG mekanizması uygular. Bir node, erişmek istediği başka bir node'a unicast PING gönderebilir. Geçerli PING mesajını alan hedef node, aynı Ping ID değerini içeren PONG mesajını kaynak node'a geri göndermek zorundadır.

PING ve PONG uygulama REQUEST/RESPONSE mekanizmasından ayrı bir network-management mekanizmasıdır. Her iki mesaj da aşağıdaki alanları kullanır:

```
Priority      = DEOS_PRIO_NETWORK
Message Class = DEOS_CLASS_NETWORK
Service       = DEOS_SERVICE_SYSTEM
```

PING için `Command = DEOS_CMD_SYSTEM_PING`; PONG için `Command = DEOS_CMD_SYSTEM_PONG` kullanılır.

17.1 PING Mesaj Formatı

PING mesajının uygulama veri alanı aşağıdaki gibi tanımlanır:

Byte	Alan	Tip	Açıklama
0	Protocol Version	`uint8_t`	DEOS v1.1 için `0x11`
1	Sequence	`uint8_t`	Gönderen node'un TX sequence değeri
2	Command	`uint8_t`	`DEOS_CMD_SYSTEM_PING` = `0x08`
3	Length	`uint8_t`	`0x04`
4–7	Ping ID	`uint32_t`	Gönderen tarafından üretilen correlation değeri

Toplam semantic CAN-FD data length 8 byte'tır. Ping ID, DEOS genel byte-order kuralına uygun olarak Little-Endian taşınır. Gönderen node, aynı anda birden fazla PING işlemi yürütüyorsa aktif işlemleri ayırt edecek Ping ID değerleri kullanmalıdır.

PING CAN Identifier alanları:

```
Priority      = DEOS_PRIO_NETWORK
Message Class = DEOS_CLASS_NETWORK
Service       = DEOS_SERVICE_SYSTEM
Destination   = ping atılan node
Source        = ping gönderen node
```

17.2 PONG Mesaj Formatı

PONG mesajı, alınan PING mesajındaki 32-bit Ping ID değerini değiştirmeden geri taşır.

Byte	Alan	Tip	Açıklama
0	Protocol Version	`uint8_t`	DEOS v1.1 için `0x11`
1	Sequence	`uint8_t`	PONG gönderen node'un kendi TX sequence değeri
2	Command	`uint8_t`	`DEOS_CMD_SYSTEM_PONG` = `0x09`
3	Length	`uint8_t`	`0x04`
4–7	Ping ID	`uint32_t`	PING'den aynen echo edilen correlation değeri

PONG CAN Identifier alanları:

```
Priority      = DEOS_PRIO_NETWORK
Message Class = DEOS_CLASS_NETWORK
Service       = DEOS_SERVICE_SYSTEM
Destination   = PING mesajının Source Node değeri
Source        = PING'i alan hedef node
```

PONG mesajındaki Sequence alanı PING mesajındaki Sequence değerini kopyalamaz. Her node kendi transmit sequence sayacını kullanmaya devam eder. PING ile PONG arasındaki transaction correlation yalnızca Ping ID üzerinden yapılır.

17.3 Zorunlu Node Davranışı

Geçerli ve kendisine yöneltilmiş bir PING alan her DEOS node'u aşağıdaki davranışı uygular:

1. PING mesajının Source Node ve Ping ID alanlarını okur.
2. Yeni bir PONG mesajı oluşturur.
3. PONG Source alanını kendi Node ID'si yapar.
4. PONG Destination alanını PING Source değerine eşitler.
5. PING içerisindeki Ping ID değerini değiştirmeden PONG payload'ına kopyalar.
6. Kendi normal TX Sequence değerini kullanarak PONG'u gönderir.

17.4 RTT Ölçümü

Round-trip time ölçümü gönderen node üzerinde yapılır. PING gönderildiği anda yerel zaman kaydedilir; aynı Ping ID'yi taşıyan PONG alındığında iki yerel zaman arasındaki fark RTT olarak hesaplanır:

$$RTT = T_{pong} - T_{ping}$$

Bu yöntem node'lar arasında saat senkronizasyonu gerektirmez. Timeout süresi transport veya uygulama gereksinimine göre ayrıca tanımlanabilir; v0.3 kapsamında tek bir global timeout süresi dondurulmamıştır.

17.5 Broadcast PING Kuralı

`DEOS_CMD_SYSTEM_PING` mesajı v1.1 protokolünde yalnızca unicast Destination Node ile kullanılmalıdır. `Destination = DEOS_NODE_BROADCAST` ile PING gönderimi geçersizdir. Bu kural, çok sayıda node'un aynı anda PONG üretmesinden doğacak response burst trafiğini önlemek için tanımlanmıştır.

Bir uygulama tüm node'ların erişilebilirliğini taramak istiyorsa her hedef Node ID için ayrı unicast PING işlemi başlatmalıdır.

17.6 HEARTBEAT ile PING–PONG Ayrımı

HEARTBEAT, bir node'un periyodik veya olay tabanlı olarak kendi yaşam bilgisini yayınlamasıdır. PING–PONG ise belirli bir node'un başka belirli bir node'a aktif erişilebilirlik sorgusu yapmasıdır. Bu iki mekanizma birbirinin yerine kullanılmaz.

18 Mantıksal Node Kimliği Kuralı

CAN hattında her Source ve Destination Node ID bağımsız bir DEOS protokol kimliğidir. Aynı fiziksel cihaz üzerinde birden fazla mantıksal Node ID bulunabilse bile, bir mesaj hangi Node ID adına üretiliyorsa CAN Identifier içerisindeki Source alanında o Node ID kullanılmalıdır.

Bir node başka bir node'un kimliğiyle cevap üretmemelidir. Örneğin `Destination = DEOS_NODE_BMS_MAIN` olan geçerli bir PING'e verilen PONG mesajında `Source = DEOS_NODE_BMS_MAIN` kullanılmalıdır.

Bu kural fiziksel yerleşimden bağımsızdır; CAN hattındaki diğer katılımcılar mantıksal Node ID'leri bağımsız haberleşme uçları olarak değerlendirir.

19 Örnek Mesaj Kimlikleri

19.1 Direksiyon Hedef Açısı

```
Source        = MAIN_STM32
Destination   = STEERING
Priority      = CONTROL
Message Class = COMMAND
Service       = STEERING
Command       = SET_TARGET_ANGLE
Payload       = int16_t target_angle, scale 0.01 deg
```

19.2 Direksiyon PID Kp Güncellemesi

```
Source        = MAIN_STM32
Destination   = STEERING
Priority      = CONFIG
Message Class = CONFIG
Service       = STEERING
Command       = SET_PARAMETER
Parameter ID  = STEERING_PID_KP
Value         = ilgili parameter taniminda belirtilen wire type
```

19.3 Direksiyon Kalibrasyonu Başlatma

```
Source        = MAIN_STM32
Destination   = STEERING
Message Class = COMMAND
Service       = CALIBRATION
Command       = START
```

19.4 BMS Durum Mesajı

```
Source        = BMS_MAIN
Destination   = MAIN_STM32
Message Class = STATUS
Service       = BMS
Command       = STATUS
```

Gateway kullanılması bu mantıksal kimliği değiştirmez.

19.5 Aydınlatma Komutu (Ön Farları Açma)

```
Source        = DEOS_NODE_GROUND_CONTROL
Destination   = DEOS_NODE_MAIN_STM32
Priority      = DEOS_PRIO_CONTROL
```

```
Message Class = DEOS_CLASS_COMMAND
Service       = DEOS_SERVICE_LIGHTING
Command       = DEOS_CMD_LIGHTING_TURN_ON
Length        = 0x01
Payload       = [0x01] (Light ID: HEADLIGHT)
```

19.6 Ground Control'dan Textual Node'a PING

```
Source        = DEOS_NODE_GROUND_CONTROL
Destination   = DEOS_NODE_TEXTUAL
Priority      = DEOS_PRIO_NETWORK
Message Class = DEOS_CLASS_NETWORK
Service       = DEOS_SERVICE_SYSTEM
Command       = DEOS_CMD_SYSTEM_PING
Ping ID       = 0x00001234
```

Textual Node aşağıdaki PONG mesajını üretir:

```
Source        = DEOS_NODE_TEXTUAL
Destination   = DEOS_NODE_GROUND_CONTROL
Priority      = DEOS_PRIO_NETWORK
Message Class = DEOS_CLASS_NETWORK
Service       = DEOS_SERVICE_SYSTEM
Command       = DEOS_CMD_SYSTEM_PONG
Ping ID       = 0x00001234
```

20 İsimlendirme Standardı

Tüm protokol sabitlerinde ortak prefix yapısı kullanılır:

```
DEOS_NODE_...
DEOS_PRIO_...
DEOS_CLASS_...
DEOS_SERVICE_...
DEOS_CMD_<SERVICE>_...
DEOS_PARAM_<SERVICE>_...
DEOS_STATE_...
DEOS_MODE_...
```

Örnekler:

```
DEOS_NODE_TEXTUAL
DEOS_NODE_GROUND_CONTROL
DEOS_NODE_MAIN_STM32
DEOS_NODE_MICRO_ROS
DEOS_NODE_STEERING
DEOS_CLASS_CONFIG
DEOS_SERVICE_STEERING
DEOS_CMD_STEERING_SET_TARGET_ANGLE
DEOS_CMD_SET_PARAMETER
DEOS_PARAM_STEERING_PID_KP
DEOS_STATE_ACTIVE
DEOS_MODE_AUTONOMOUS
```

21 Özet Registry

Mevcut CAN-FD protokol kararları aşağıdaki şekilde özetlenebilir:

- Ana araç içi haberleşme hattı CAN-FD'dir.
- 29-bit Extended CAN Identifier kullanılır.
- CAN-FD frame'lerinde BRS kullanılır.
- Priority, Message Class, Service, Destination ve Source alanları CAN Identifier içerisinde taşınır.
- Node ID, sistem genelinde kullanılan 8-bit mantıksal protokol kimliğidir.
- Aynı fiziksel cihaz birden fazla mantıksal Node ID barındırabilir; CAN hattında her biri bağımsız kimliktir.
- DEOS Protocol v1.1 sürüm değeri `0x11`'dir.
- Ortak DEOS data header'ı Version, Sequence, Command ve Length olmak üzere 4 byte'tır.
- Command-specific maksimum payload uzunluğu 60 byte'tır.
- Length, yalnızca geçerli command-specific payload byte sayısını belirtir.
- DLC fiziksel CAN-FD data field boyutunu; Length ise semantic DEOS payload boyutunu ifade eder.
- DLC yuvarlaması sonucu oluşan kullanılmayan byte'lar `0x00` padding olarak gönderilir.
- Uygulama seviyesinde ek CRC veya ayrıca sync header kullanılmaz.
- Sequence alanı Source Node başına kullanılır; v1.1'de gap/duplicate kontrolü zorunlu değildir.
- REQUEST, RESPONSE ve CONFIG birbirinden ayrı Message Class değerleridir.
- CONFIG ayrı bir Service değildir.
- Parameter ID 16-bit'tir ve Service namespace'i içinde anlam kazanır.
- Varsayılan çok baytlı wire byte sırası Little-Endian'dır.
- Fiziksel değerlerde varsayılan yaklaşım ölçekli integer kullanımıdır.
- PING/PONG, `DEOS_CLASS_NETWORK / DEOS_SERVICE_SYSTEM` altında tanımlıdır.
- PING ve PONG command payload'u 4-byte Ping ID içerir ve Length alanı `0x04`'tür.
- PING yalnız unicast olarak geçerlidir; Broadcast PING'e PONG üretilmez.
- Bir cevabın Source Node değeri, cevabı üreten mantıksal node kimliği olmalıdır.

22 Açık Konular ve Sonraki ICD Revizyonları

Mesaj bazında henüz dondurulmamış başlıca CAN-FD protokol alanları şunlardır:

- Her mesajın tam command payload byte haritası,
- Her komutun minimum ve maksimum fiziksel sınırları,
- Her komutun periyodu ve timeout değeri,
- Safety mesajlarının nihai priority seviyeleri,
- Kalibrasyon payload ayrıntıları,
- BMS status mesajının nihai sinyal listesi,
- Parameter value type bilgisinin mesaj içinde mi yoksa registry'de mi tutulacağı,
- Future firmware update / bootloader mesajları,
- Sequence alanının gelecekteki kesin semantiği,
- Nominal CAN bitrate ve CAN-FD data-phase bitrate değerleri,
- Genel broadcast mesajlarının hangi servis ve komutlarda geçerli olacağı,
- PING/PONG timeout ve retry politikaları.

Bu alanlar sonraki ICD revizyonlarında ayrıntılandırılacaktır. v1.1'de tanımlanan 29-bit Identifier yapısı, 4-byte common data header, Length alanı, 60-byte maksimum command payload, BRS kullanımı ve mantıksal Node ID kuralları sonraki revizyonların referans temelidir.